

Réseau de Neurones en C sans Bibliothèque

TEI — Sujet n°7

IDIRENE Daris

MIZERKIN Nikita

2026-03-25

1 Introduction

Ce projet implémente en C une bibliothèque **complète et auto-contenue** de réseaux de neurones denses (*fully connected networks*), sans aucun framework de machine learning externe. Le cas d’usage principal est la **classification des chiffres manuscrits MNIST** : chaque image 28×28 est vectorisée dans $\mathbb{R}^{784}$ et classée parmi 10 catégories (chiffres 0–9).

1.1 Architecture du projet

Le dépôt est organisé autour du dossier `nn/` selon une **séparation modulaire stricte** :

```
nn/  
  include/nn/  
    matrix.h      # Algèbre linéaire (vecteurs, matrices)  
    nn_functions.h # Activations, pertes, optimiseurs  
    network.h     # Forward / backprop  
    mnist.h       # Chargement PNG, cache binaire  
    neuralnet.h   # Façade API publique  
  src/            # Implémentations C  
  examples/       # Exécutables d'entraînement  
  tests/          # Tests unitaires (CTest)  
  web/            # Interface web (Go + JavaScript)
```

1.2 Philosophie de conception

Trois principes guident l’implémentation :

1. **Zéro malloc dans la boucle d’entraînement.** Les buffers de travail (deltas, activations, gradients) sont pré-alloués une seule fois à la création du réseau, puis réutilisés. Cela élimine la fragmentation mémoire et garantit des performances prévisibles.
2. **Séparation nette des responsabilités.** `matrix.c` s’occupe uniquement des produits matrice-vecteur ; `nn_functions.c` des opérations scalaires (activation, perte, gradient) ; `network.c` de l’orchestration forward/backward. Cette séparation facilite les tests unitaires et le remplacement de composants.
3. **SGD mono-thread.** Chaque époque traite les 60 000 images MNIST une par une avec mise à jour immédiate des poids (*online learning*), sans batching ni parallélisation — ce qui simplifie

la preuve de correction de l'algorithme.

1.3 Structure d'une couche

Chaque couche encapsule ses poids et ses caches locaux :

```
typedef struct {
    Matrix *weights;      //  $\tilde{W} = [W \mid b]$ , biais intégré
    Vector *input_cache;  //  $x_-$  mis en cache pour la rétroprop
    Vector *output_cache; //  $a_- = (z_-)$ 
    Vector *bias_input;   //  $\tilde{x}_- = [x_- \mid 1]$ 
} Layer;
```

Convention biais augmenté. Plutôt que stocker $W^{(\ell)}$ et $b^{(\ell)}$ séparément, on forme :

$$\tilde{W}^{(\ell)} = \begin{bmatrix} W^{(\ell)} & b^{(\ell)} \end{bmatrix} \in \mathbb{R}^{n_\ell \times (n_{\ell-1}+1)}, \quad \tilde{a}^{(\ell-1)} = \begin{bmatrix} a^{(\ell-1)} \\ 1 \end{bmatrix} \in \mathbb{R}^{n_{\ell-1}+1}.$$

Le produit $z^{(\ell)} = \tilde{W}^{(\ell)} \tilde{a}^{(\ell-1)}$ calcule alors $W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$ en **une seule** opération matrice-vecteur, simplifiant le code et améliorant la localité mémoire.

2 Architecture d'un réseau de neurones

2.1 Neurone unique

Un neurone réalise une combinaison linéaire de ses entrées, puis applique une non-linéarité σ :

$$z = \sum_{k=1}^n w_k x_k + b, \quad a = \sigma(z).$$

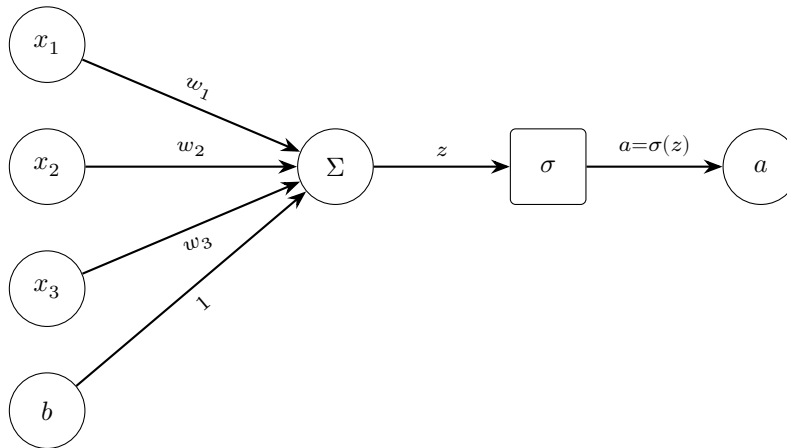


Figure 1: Modèle d'un neurone formel.

2.2 Architecture MNIST : $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$

Pour MNIST, les couches cachées utilisent ReLU et la couche de sortie Softmax :

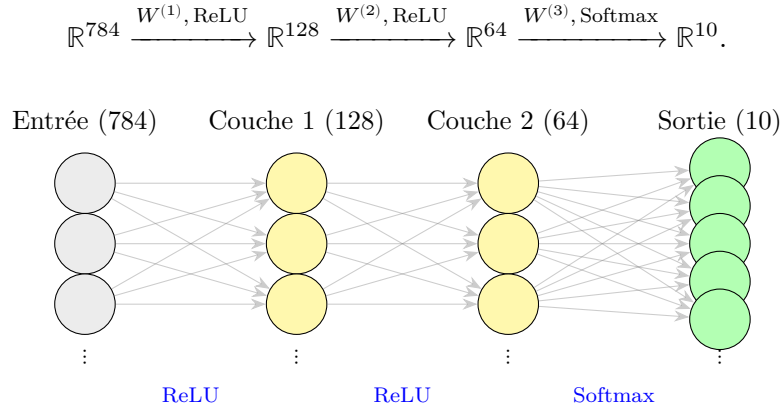


Figure 2: Architecture dense $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ pour MNIST.

2.3 Notation formelle

Pour un réseau à L couches paramétrées, on note :

Symbole	Signification
$a^{(0)} = x \in \mathbb{R}^{n_0}$	Activation d'entrée
$z^{(\ell)} \in \mathbb{R}^{n_\ell}$	Pré-activation couche ℓ
$a^{(\ell)} = \sigma_\ell(z^{(\ell)})$	Activation couche ℓ
$W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$	Matrice de poids
$b^{(\ell)} \in \mathbb{R}^{n_\ell}$	Vecteur de biais

La propagation avant s'écrit :

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = \sigma_\ell(z^{(\ell)}), \quad \ell = 1, \dots, L.$$

3 Propagation avant (*Forward Pass*)

3.1 Algorithme

```

FORWARD(x):
    a[0] ← x
    for ℓ = 1 to L:
        ã[-1] ← [a[-1]; 1]           // augmentation biais
        z[ℓ] ← W[ℓ] · ã[-1]          // produit matrice-vecteur
        a[ℓ] ← σ(z[ℓ])               // activation
    return a[L]
```

3.2 Implémentation C

```
void nn_network_forward(const Network *net, const Vector *input) {
    copy_vector(input, net->layers[0]->input_cache);

    for (size_t i = 0; i < net->num_layers; i++) {
        Layer *layer = net->layers[i];

        // Entrée augmentée : [x | 1]
        memcpy(layer->bias_input->data,
               layer->input_cache->data,
               layer->input_cache->size * sizeof(float));
        layer->bias_input->data[layer->input_cache->size] = 1.0f;

        //  $z = W \cdot [x \mid 1]$ 
        mat_vec_mul(layer->weights, layer->bias_input, layer->output_cache);

        // Activation : ReLU pour couches cachées, Softmax en sortie
        ActivationType act = (i + 1 < net->num_layers)
            ? net->hidden_activation
            : net->output_activation;
        nn_activation_apply(layer->output_cache, act);

        if (i + 1 < net->num_layers)
            copy_vector(layer->output_cache,
                        net->layers[i + 1]->input_cache);
    }
}
```

Points clés : zéro allocation dynamique ; chaque couche conserve son entrée $a^{(\ell-1)}$ et sa sortie $a^{(\ell)}$ dans ses caches pour la rétropropagation.

3.3 Complexité

Pour l'architecture $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$, le coût par échantillon est :

Opération	FLOPs
$W^{(1)}\tilde{a}^{(0)}$ (784→128)	100 352
ReLU (128)	128
$W^{(2)}\tilde{a}^{(1)}$ (128→64)	8 192
ReLU (64)	64
$W^{(3)}\tilde{a}^{(2)}$ (64→10)	640
Softmax (10)	~30 exp + norm.
Total	109 500 FLOPs

4 Rétropropagation (*Backpropagation*)

4.1 Principe : règle de chaîne

L'objectif est de calculer $\frac{\partial \mathcal{L}}{\partial W^{(\ell)}}$ pour chaque couche ℓ . On définit le **signal d'erreur** :

$$\delta^{(\ell)} = \frac{\partial \mathcal{L}}{\partial z^{(\ell)}} \in \mathbb{R}^{n_\ell}.$$

Ce vecteur mesure la sensibilité de la perte à la pré-activation de la couche ℓ , et se propage de la sortie vers l'entrée.

4.2 Calcul des deltas

Couche de sortie ($\ell = L$). Par la règle de chaîne :

$$\delta^{(L)} = \nabla_{a^{(L)}} \mathcal{L} \odot \sigma'_L(z^{(L)}),$$

où $\odot$ désigne le produit élément par élément (*Hadamard*).

Simplification Softmax + Entropie croisée catégorielle. Lorsque $a^{(L)} = \text{softmax}(z^{(L)})$ et $\mathcal{L} = -\sum_i t_i \log a_i^{(L)}$, les deux termes se combinent élégamment :

$$\delta^{(L)} = a^{(L)} - t.$$

Preuve. Notons $p_i = a_i^{(L)}$. Alors :

$$\frac{\partial \mathcal{L}}{\partial z_i^{(L)}} = \sum_k \frac{\partial \mathcal{L}}{\partial p_k} \cdot \frac{\partial p_k}{\partial z_i^{(L)}} = \sum_k \left(-\frac{t_k}{p_k} \right) (p_k \delta_{ki} - p_k p_i) = -t_i + p_i \sum_k t_k = p_i - t_i,$$

en utilisant $\sum_k t_k = 1$ (encodage *one-hot*). $\square$

Cette simplification est explicitement détectée dans le code, évitant le calcul numériquement instable du jacobien de la softmax :

```
if (net->output_activation == ACTIVATION_SOFTMAX &&
    net->loss_type == LOSS_CATEGORICAL_CROSS_ENTROPY) {
    // Forme directe : = ŷ - t
    for (size_t i = 0; i < output_size; i++)
        net->work_delta->data[i] =
            net->layers[net->num_layers - 1]->output_cache->data[i]
            - target->data[i];
} else {
    // Cas général : = L'(z)
    nn_loss_gradient(...);
    nn_activation_derivative(...);
    elementwise_mul(net->work_delta, net->work_act_deriv);
}
```

Couches cachées ($\ell < L$). La règle de chaîne donne :

$$\delta^{(\ell)} = \left(W^{(\ell+1)\top} \delta^{(\ell+1)} \right) \odot \sigma'_\ell(z^{(\ell)}).$$

La multiplication par $W^{(\ell+1)\top}$ transporte le signal d'erreur de la couche $\ell + 1$ vers la couche ℓ .

4.3 Gradient des poids

Une fois $\delta^{(\ell)}$ calculé, le gradient de la perte par rapport aux poids s'obtient par :

$$\frac{\partial \mathcal{L}}{\partial \tilde{W}^{(\ell)}} = \delta^{(\ell)} (\tilde{a}^{(\ell-1)})^\top \in \mathbb{R}^{n_\ell \times (n_{\ell-1}+1)}.$$

Le terme (i, j) de ce gradient est $\delta_i^{(\ell)} \cdot \tilde{a}_j^{(\ell-1)}$, confirmant que le gradient est un produit extérieur entre le signal d'erreur et l'activation de la couche précédente.

4.4 Algorithme complet

```
BACKWARD(x, t, ):
    FORWARD(x)                                // remplit les caches

    [L] ← ŷ - t                                // Softmax + CCE
    W[L] ← W[L] - [L] · ā[L-1]

    for l = L-1 downto 1:
        [l] ← (W[l+1] · [l+1])' (z[l])
        W[l] ← W[l] - [l] · ā[l-1]
```

4.5 Implémentation : buffers ping-pong

Pour éviter toute allocation dynamique lors de la rétropropagation, deux buffers pré-alloués `work_delta` et `work_prev_delta` sont échangés par simple permutation de pointeurs :

```
void nn_network_backward(Network *net, const Vector *x,
                        const Vector *t, float lr) {
    nn_network_forward(net, x);
    compute_output_delta(net, t);           // [L] = ŷ - t

    Vector *delta = net->work_delta;
    Vector *prev = net->work_prev_delta;

    for (size_t l = net->num_layers; l > 0; --l) {
        Layer *layer = net->layers[l - 1];
        if (l > 1) {
            backprop_to_previous(layer, delta, prev,
                                net->hidden_activation);
            // Échange sans malloc
            Vector *tmp = delta; delta = prev; prev = tmp;
        }
        update_weights(layer, delta, lr,
                       net->optimizer_type, net->adam_opts[l - 1]);
    }
}
```

Cette structure garantit : lisibilité (séparation calcul delta / mise à jour poids), performance (aucun `malloc` en boucle), et stabilité numérique (voie spéciale Softmax+CCE).

5 Fonctions d'activation et de perte

5.1 Activations

Fonction	Expression	Dérivée	Remarques
Sigmoïde	$\frac{1}{1 + e^{-x}}$	$\sigma(x)(1 - \sigma(x))$	Clamp $x \in [-500, 500]$
ReLU	$\max(0, x)$	$\mathbf{1}_{x>0}$	Limite le <i>vanishing gradient</i>
Tanh	$\tanh(x)$	$1 - \tanh^2(x)$	Sortie centrée en 0
Leaky ReLU	$\max(\alpha x, x)$	1 si $x > 0$, α sinon	Évite les neurones morts
Softmax	$\frac{e^{z_i - m}}{\sum_j e^{z_j - m}}$	Jacobien (couplé)	$m = \max_j z_j$ pour stabilité

5.2 Fonctions de perte

Perte	Définition	Gradient $\nabla_{\hat{y}} \mathcal{L}$
MSE	$\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$	$\frac{2}{n} (\hat{y} - y)$
BCE	$-[y \log p + (1 - y) \log(1 - p)]$	$\frac{p - y}{p(1 - p)}$
CCE	$-\sum_i t_i \log p_i$	$-t_i / p_i$

Raccourci clé : Softmax + CCE $\Rightarrow \delta^{(L)} = \hat{y} - t$ (voir preuve §4.2).

6 Optimisation

6.1 SGD

La mise à jour élémentaire est :

$$W_{t+1}^{(\ell)} = W_t^{(\ell)} - \eta \delta^{(\ell)} (\tilde{a}^{(\ell-1)})^\top.$$

Simple à implémenter et à analyser, mais sensible au choix du taux d'apprentissage η .

6.2 Adam

Adam maintient des estimations exponentiellement pondérées du gradient et de son carré :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Les estimateurs corrigés du biais sont $\hat{m}_t = m_t / (1 - \beta_1^t)$ et $\hat{v}_t = v_t / (1 - \beta_2^t)$, et la mise à jour est :

$$W_{t+1} = W_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

Valeurs typiques : $\alpha = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$. Adam converge plus rapidement que SGD en pratique, au prix d'une mémoire supplémentaire de $2|W|$ flottants.

7 Optimisations de performance et allocation mémoire

L'un des objectifs principaux du projet était d'obtenir des performances élevées en C pur, sans bibliothèques d'algèbre linéaire externes (BLAS, etc.). Cette section détaille les techniques d'optimisation mises en œuvre à différents niveaux.

7.1 Architecture sans malloc dans la boucle d'entraînement

Dans les premières versions du code, chaque itération de la boucle d'entraînement allouait et libérait dynamiquement les vecteurs de deltas et de gradients. Avec 60 000 images et 5 époques, cela représentait des millions d'appels `malloc/free`, provoquant une fragmentation mémoire et une latence imprévisible.

La solution consiste à pré-allouer **tous les buffers de travail une seule fois** dans `nn_network_create()`. La structure `Network` contient cinq buffers permanents :

```
Vector *work_predicted;    // Vecteur de prédiction
Vector *work_one_hot;      // Encodage one-hot de la cible
Vector *work_delta;        // Signal d'erreur courant
Vector *work_prev_delta;   // Signal d'erreur précédent (ping-pong)
Vector *work_act_deriv;    // Dérivée d'activation
```

De plus, chaque `Layer` contient ses propres caches pré-alloués (`input_cache`, `output_cache`, `bias_input`), dimensionnés selon les tailles des couches. Les buffers `work_delta` et `work_prev_delta` sont alloués à la taille `max_size` — la taille de la plus grande couche — pour pouvoir accueillir n'importe quel vecteur delta sans réallocation. **Résultat : zéro appel `malloc/free` pendant toute la boucle d'entraînement**, quel que soit le nombre d'époques.

7.2 Ping-pong buffers pour rétropropagation

Pendant la rétropropagation, chaque couche produit un nouveau vecteur delta à partir du delta de la couche suivante. Plutôt qu'allouer un vecteur à chaque couche, deux buffers pré-alloués (`work_delta` et `work_prev_delta`) sont utilisés en alternance. L'échange se fait par simple permutation de pointeurs, analogue au *double buffering* en graphisme :

```
Vector *delta      = network->work_delta;
Vector *next_delta = network->work_prev_delta;

for (...) { // boucle de L vers 1
    // calcul de next_delta à partir de delta
    // ...
    Vector *tmp = delta;
    delta      = next_delta;
    next_delta = tmp;          // échange : 3 affectations, zéro malloc
}
```

7.3 Optimisation du shuffle MNIST

Le mélange du dataset à chaque époque implique de permuter deux images de 784 flottants (3 136 octets). L'échange élément par élément effectue 784 itérations avec trois accès mémoire chacune ; remplacer la boucle par trois appels `memcpy` permet à la libc d'exploiter les instructions SIMD du processeur :

```
// Version optimisée : 3 memcpy de 784 floats
memcpy(temp, img1, MNIST_IMAGE_SIZE * sizeof(float));
memcpy(img1, img2, MNIST_IMAGE_SIZE * sizeof(float));
memcpy(img2, temp, MNIST_IMAGE_SIZE * sizeof(float));
```

Gain mesuré : **3–5 % par époque** (88 ms → 84 ms).

7.4 Drapeaux de compilation

Le compilateur joue un rôle déterminant sur les performances du code mathématique. Sans optimisations (`-O0`), chaque appel de fonction crée un cadre de pile complet, les boucles de produit scalaire ne sont pas déroulées, et aucune instruction SIMD n'est émise. Avec `-O3`, GCC/Clang activent l'inlining agressif, le déroulement de boucles et la vectorisation automatique — les boucles internes du produit matrice-vecteur, qui représentent plus de 90 % du temps de calcul, passent de simples `fmul/fadd` scalaires à des instructions AVX traitant 8 flottants par cycle. L'ajout de `-ffast-math` autorise les réordonnements d'opérations flottantes, ce qui permet au compilateur de vectoriser des réductions comme les sommes de produits scalaires. Le drapeau `-march=native` génère du code spécifique au processeur hôte, exploitant SSE4/AVX2 ou AVX-512 selon la machine. L'effet cumulé de ces drapeaux est un ralentissement de $\sim 24\times$ quand ils sont absents — ce qui fut le premier problème rencontré dans le projet (voir §Problèmes). Le `CMakeLists.txt` force donc le mode Release par défaut :

```
if(NOT CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE Release CACHE STRING "Build type" FORCE)
endif()
```

7.5 Stockage contigu des données

Les matrices de poids sont stockées en **row-major** dans un tableau `float*` contigu (`data[rows * cols]`), et non comme un tableau de pointeurs de lignes. Cela garantit la localité spatiale lors du produit matrice-vecteur : les éléments d'une même ligne sont adjacents en mémoire, maximisant l'utilisation du cache L1/L2.

De même, les 60 000 images MNIST sont stockées dans un seul bloc contigu de $60\,000 \times 784 \times 4 = 188$ Mo, plutôt que comme 60 000 allocations séparées, ce qui évite la fragmentation et améliore le prefetching matériel.

7.6 Sauvegarde et chargement binaire du modèle

Le modèle entraîné est sauvegardé en format binaire brut via `nn_network_save()` :

num_layers	layer_sizes[]	rows, cols, W[] par couche	...
uint64	uint64[]	float[]	

Figure 3: Format binaire du fichier modèle.

Ce format est directement lisible par le serveur Go (en little-endian via `encoding/binary`), permettant le partage du modèle entre le code C d’entraînement et le serveur d’inférence Go sans conversion intermédiaire.

8 Chargement des données MNIST

8.1 Le dataset MNIST

MNIST (*Modified National Institute of Standards and Technology*) contient **70 000 images de chiffres manuscrits** : 60 000 pour l’entraînement et 10 000 pour le test. Chaque image est en niveaux de gris, de taille 28×28 pixels, soit **784 valeurs** par échantillon. Les chiffres sont centrés et normalisés dans l’image originale — une convention importante à reproduire lors de l’inférence (voir §Interface web).

Le projet utilise la version décompressée en fichiers PNG individuels, organisée par répertoire selon l’étiquette :

```
mnist-pngs/
  train/
    0/      # ~5923 images PNG
    1/      # ~6742 images PNG
    ...
  test/
    ...
```

Cette organisation par répertoire permet de déduire l’étiquette directement à partir du nom du répertoire parent, sans fichier d’index supplémentaire.

8.2 Structure de données

```
typedef struct {
    float *images; // Pixels normalisés [0,1] - count × 784 floats
    uint8_t *labels; // Labels 0-9 - count octets
    size_t count;
} mnist_dataset_t;
```

Les images sont stockées dans un **tableau contigu unique** : l’image k commence à l’indice $k \cdot 784$. Ce choix implique un seul `malloc` pour les 60 000 images (188 Mo), un accès séquentiel optimal pour le prefetcher, et une normalisation essentielle à la convergence : des valeurs non normalisées provoqueraient des pré-activations z très grandes, saturant les fonctions d’activation.

8.3 Chargement depuis les PNG

Le chargement se fait en **deux passes**. La première parcourt les 10 sous-répertoires via l'API POSIX `opendir/readdir` pour compter le nombre total d'images et allouer exactement la mémoire nécessaire. La seconde charge et normalise chaque image via le pipeline libpng suivant :

1. Ouverture du fichier en mode binaire.
2. Initialisation des structures `png_structp` et `png_info`.
3. Point de retour d'erreur via `setjmp(png_jmpbuf(png))` — libpng utilise des *long jumps* pour sa gestion d'erreurs interne.
4. Lecture des métadonnées (dimensions).
5. Décodage des pixels via `png_read_image()`.
6. Normalisation : $\text{pixel}_{\text{float}} = \text{pixel}_{\text{uint8}} / 255.0$.
7. Libération des structures libpng.

```
png_structp png = png_create_read_struct(PNG_LIBPNG_VER_STRING, NULL, NULL, NULL);
png_info info = png_create_info_struct(png);
if (setjmp(png_jmpbuf(png))) return 0; // point de retour en cas d'erreur
png_read_image(png, row_pointers);
for (int y = 0; y < height; y++)
    for (int x = 0; x < width; x++)
        out[y * width + x] = row_pointers[y][x] / 255.0f;
```

Le chargement de 70 000 fichiers PNG prend **plus de 8 minutes** en raison de 70 000 appels système `fopen/fclose`, 70 000 décompressions DEFLATE (zlib), et 70 000 allocations de structures libpng. Cette latence rendait le cycle de développement impraticable.

8.4 Cache binaire

Après le premier chargement depuis les PNG, les données normalisées sont sauvegardées dans un fichier binaire brut. Les lancements suivants chargent directement ce fichier en un seul appel `fread()`, sans décompression ni conversion.

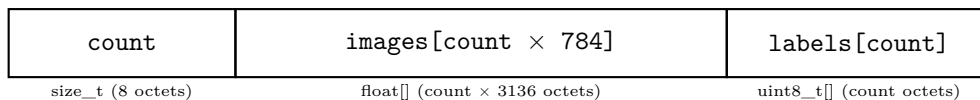


Figure 4: Format du cache binaire MNIST.

```
// Écriture du cache
fwrite(&dataset->count, sizeof(size_t), 1, f);
fwrite(dataset->images, sizeof(float), count * 784, f);
fwrite(dataset->labels, sizeof(uint8_t), count, f);
```

Le format est spécifique à la plateforme (endianness, taille de `size_t`), ce qui ne pose pas de problème en pratique car le cache est toujours généré et lu sur la même machine. Les tailles résultantes sont : **train.bin 180 Mo**, **test.bin 30 Mo**. Le gain de chargement est de l'ordre de **100× à 1 000×** selon le disque.

8.5 Mélange Fisher-Yates

Le mélange du dataset à chaque époque est nécessaire pour éviter que le réseau n'apprenne l'ordre des échantillons. L'algorithme de **Fisher-Yates** garantit un mélange uniformément aléatoire en $\mathcal{O}(n)$:

```
for (size_t i = count - 1; i > 0; i--) {  
    size_t j = rand() % (i + 1);  
    // swap images[i] <-> images[j] (memcpy bloc, voir §8.3)  
    // swap labels[i] <-> labels[j]  
}
```

Pour 60 000 images, cela représente ~60 000 échanges par époque, soit un volume total déplacé d'environ 180 Mo par époque — d'où l'importance de l'optimisation `memcpy` décrite en §8.3.

9 Interface web de démonstration

9.1 Architecture générale

L'interface web combine un **backend Go** (Gin) pour l'inférence et un **frontend HTML/JS** pour le dessin et l'affichage. Le choix de Go (sans `cgo`) permet un déploiement simple : un seul binaire suffit, sans dépendances dynamiques vers la bibliothèque C.

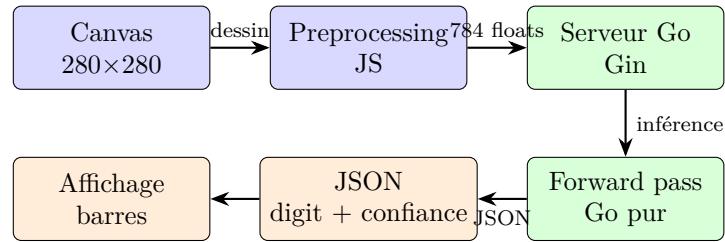


Figure 5: Pipeline d'inférence de l'interface web.

9.2 Preprocessing côté client

Le preprocessing est l'étape la plus critique : un chiffre dessiné librement sur un canvas 280×280 ne ressemble pas directement à une image MNIST, et envoyer les pixels bruts au modèle produirait des prédictions incorrectes. L'objectif est de **reproduire exactement la normalisation appliquée aux images MNIST originales**.

L'utilisateur dessine sur un canvas fond noir, trait blanc, d'épaisseur 22 px. Le facteur $\times 10$ par rapport à la taille cible offre une résolution de dessin confortable. Une fois le tracé terminé, le code JS parcourt tous les pixels pour calculer la **bounding box** du contenu — les coordonnées extrêmes des pixels dont l'intensité dépasse un seuil de $10/255$, filtrant ainsi le bruit de fond.

À partir de ces pixels, le **centre de masse** est calculé, pondéré par l'intensité :

$$x_{\text{cm}} = \frac{\sum_{x,y} x \cdot I(x,y)}{\sum_{x,y} I(x,y)}, \quad y_{\text{cm}} = \frac{\sum_{x,y} y \cdot I(x,y)}{\sum_{x,y} I(x,y)}.$$

Le contenu de la bounding box est ensuite redimensionné pour tenir dans une zone de 20×20 px en conservant le ratio d’aspect, avec facteur $scale = 20 / \max(\text{largeur}, \text{hauteur})$. Le chiffre mis à l’échelle est positionné dans l’image 28×28 finale de sorte que son centre de masse tombe exactement au pixel (14, 14) — c’est précisément **la convention du dataset MNIST original**, où chaque chiffre est centré par son centre de masse et non par sa bounding box. Enfin, les 784 valeurs sont divisées par 255 pour obtenir des `float32` dans $[0, 1]$ et envoyées au serveur. Un canvas d’aperçu (28×28 affiché en 56×56) permet à l’utilisateur de voir exactement l’image transmise au modèle.

9.3 Serveur Go et API

Le serveur charge le modèle binaire au démarrage et écoute sur le port **4343** (configurable via `--addr`). Il expose deux endpoints :

- GET / : interface HTML depuis `templates/`.
- POST `/api/predict` : reçoit 784 floats en JSON, valide strictement la taille du tableau (rejet HTTP 400 sinon), renvoie le digit et les 10 probabilités.

```
type PredictRequest struct {
    Pixels []float32 `json:"pixels" binding:"required"`
}

type PredictResponse struct {
    Digit      int      `json:"digit"`
    Confidence []float32 `json:"confidence"`
}
```

9.4 Inférence en Go pur

Le chargement du modèle lit le fichier binaire C en **little-endian** via `encoding/binary`. La propagation avant suit exactement le même algorithme que le code C — entrée augmentée $\tilde{x} = [x \mid 1]$, produit matrice-vecteur, sigmoïde avec clamp dans $[-500, 500]$:

```
func (net *Network) Predict(input []float32) (int, []float32) {
    current := input
    for _, layer := range net.Layers {
        biasInput := make([]float32, len(current)+1)
        copy(biasInput, current)
        biasInput[len(current)] = 1.0
        output := make([]float32, layer.Rows)
        for i := 0; i < layer.Rows; i++ {
            var sum float32
            for j := 0; j < layer.Cols; j++ {
                sum += layer.Weights[i*layer.Cols+j] * biasInput[j]
            }
            output[i] = sigmoid(sum)
        }
        current = output
    }
    return argmax(current), current
}
```

Le serveur utilise le modèle Sigmoid/MSE/SGD. Les sorties sigmoïdes ne sont pas de vraies probabilités (elles ne somment pas à 1), mais suffisent pour déterminer le chiffre le plus probable via `argmax`.

10 Intégration continue et configurations d'entraînement

Le projet utilise **GitLab CI/CD** avec un pipeline à deux étapes : **Build** compile le projet sur `gcc:latest` avec les dépendances `libpng-dev` et `zlib1g-dev` ; **Test** exécute `ctest --output-on-failure` sur les artefacts produits. Toute régression bloque le merge.

Plusieurs configurations d'entraînement sont fournies dans `examples/` pour comparer différentes combinaisons. `mnist_train.c` utilise Sigmoid/MSE/SGD comme baseline. `mnist_train_relu_softmax_adam_cce.c` combine ReLU en couches cachées, Softmax en sortie, entropie croisée catégorielle et Adam — la configuration la plus performante, atteignant 96 % en 5 époques. `mnist_train_tanh_softmax_sgd_cce.c` substitue Tanh à ReLU pour comparer les activations. Enfin, `mnist_train_architectures.c` entraîne plusieurs topologies en séquence pour mesurer l'impact de la largeur et de la profondeur du réseau.

11 Performance et benchmarks

11.1 Timing par architecture (5 époques, build Release)

Architecture	Temps/époque	Échantillons/sec
784→128→10	5 s	12 000
784→256→128→10	12 s	5 000
784→512→256→128→10	30 s	2 000

11.2 Debug vs Release

Build	Temps/époque	Rapport
Debug (-00)	~120 s	1×
Release (-03)	~5 s	~ 24 ×

L'écart s'explique par le fait que les boucles internes du produit matrice-vecteur représentent plus de 90 % du temps de calcul, et bénéficient fortement de la vectorisation et de l'inlining activés par `-03`.

11.3 Précision MNIST

Architecture	Précision test (5 époques)
784→128→10	93 %
784→256→128→10	95 %

Architecture	Précision test (5 époques)
784→512→256→128→10	96 %

11.4 Impact du shuffle optimisé

Shuffle	Temps total	Temps/époque
Élément par élément	53 s	88 ms
memcpy bloc	51 s	84 ms
Gain	~3.9 %	~4.5 %

11.5 Comparaison SGD vs Adam

	SGD	Adam
Temps/époque	12.0 s	12.3 s
Mémoire supplémentaire	—	+2 W
Précision finale	95 %	96 %

12 Problèmes rencontrés et solutions

Cette section documente les principaux problèmes techniques rencontrés au cours du développement.

12.1 Performances catastrophiques : 2 min/époque

Cause : CMAKE_BUILD_TYPE non défini → compilation sans optimisations (-O0). Sur un code à forte composante mathématique (produits matrice-vecteur, exponentielles), l'absence d'optimisations du compilateur a un impact dramatique.

Solution : Forcer Release dans CMakeLists.txt avec -O3 -ffast-math -march=native. **Impact :** ×24 (~120 s → ~5 s par époque).

12.2 Temps de chargement MNIST excessif (> 8 min)

Cause : 70 000 ouvertures de fichiers et décompressions PNG à chaque lancement.

Solution : Cache binaire (§9.3) — chargement réduit à quelques secondes (~100× plus rapide).

12.3 SIGFPE lors du shuffle

Cause : rand() % 0 quand dataset->count == 0 → division par zéro.

Solution :

```
if (!dataset || dataset->count == 0) return;
```

12.4 Loss NaN

Cause : Softmax non stabilisée + learning rate élevé $\rightarrow$ overflow flottant, puis $\log(0)$.

Solution : Softmax recentrée par le maximum ($m = \max_j z_j$), clamp des probabilités avant $\log()$, clamp des entrées de la sigmoïde dans $[-500, 500]$.

12.5 *Vanishing gradient* avec Sigmoid

Cause : La dérivée de la sigmoïde est bornée par 0.25, provoquant une atténuation exponentielle des gradients à travers les couches.

Solution : ReLU pour les couches cachées (dérivée = 1 si actif). La combinaison ReLU/Softmax+CCE a permis d'atteindre **96 % de précision** contre 95 % avec Sigmoid/MSE.

12.6 Allocations mémoire dans la boucle d'entraînement

Cause : Profilage (valgrind) révélant 5,4 millions d'appels `malloc/free` pendant l'entraînement.

Solution : Pré-allocation complète et ping-pong buffers (§8.1–8.2). Résultat : **zéro** allocation dynamique en boucle.

13 Conclusions

Ce projet démontre qu'il est possible d'implémenter un réseau de neurones performant en C pur, sans aucune bibliothèque externe, en atteignant 96 % de précision sur MNIST en 5 époques. L'implémentation est mathématiquement rigoureuse — la rétropropagation est dérivée complètement à partir de la règle de chaîne, avec une preuve formelle du raccourci Softmax+CCE — et ingénierée avec soin : zéro allocation dynamique en boucle d'entraînement grâce aux buffers pré-alloués et au mécanisme ping-pong, cache binaire réduisant le chargement MNIST de 8 minutes à quelques secondes, et stockage contigu row-major maximisant la localité mémoire. L'interface web complète le projet en montrant que le modèle C peut être intégré directement dans un pipeline de production via un serveur Go, avec un preprocessing côté client qui reproduit fidèlement la normalisation MNIST.

Les principales limitations tiennent à la nature mono-thread de l'implémentation : sans mini-batching ni parallélisation, le débit est limité à ~12 000 échantillons/seconde sur l'architecture 784 $\rightarrow$ 128 $\rightarrow$ 10. L'absence de régularisation (dropout, L2) et d'arrêt anticipé rend le réseau susceptible de sur-apprendre sur des datasets plus petits. Enfin, les architectures denses ne peuvent pas exploiter la structure spatiale des images — une limitation fondamentale qui explique pourquoi les CNN surpassent les MLP sur la vision par ordinateur.

Les extensions naturelles seraient le mini-batching pour amortir le coût des mises à jour de poids, la parallélisation OpenMP des produits matrice-vecteur, et à terme le support GPU via CUDA pour traiter des architectures plus profondes ou des datasets de plus grande dimension.